Chapter 1 解答

CH01-Q01

1. ISA 保證指令編碼與語意、程式可見 registers、addressing、例外契約等。只要處理器正確實作同一 ISA，既有 machine code 可以維持正確定義。
2. Microarchitecture 可以改變 pipeline 階數、cache、predictor、issue width、forwarding 與 stall 行為，因此 CPI、clock cycle time 與某些 timing-visible effects 會改變。
3. 同一程式的 IC 可能相近，但 new microarchitecture 提供不同 CPI；frequency 也可能不同。總時間 $IC \times CPI / f$ 由三者共同決定，因此 ISA compatibility 只保證行為，不保證效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH01-Q02

1. P 的 cycles 與時間：

$$C_P = (3.6 \times 10^8)(1.50) = 5.4 \times 10^8,$$
$$T_P = \frac{5.4 \times 10^8}{2.4 \times 10^9} = 0.225\ \text{s} = 225\ \text{ms}.$$

Q 的 cycles 與時間：

$$C_Q = (3.0 \times 10^8)(1.80) = 5.4 \times 10^8,$$
$$T_Q = \frac{5.4 \times 10^8}{2.8 \times 10^9} \approx 0.192857\ \text{s} = 192.857\ \text{ms}.$$

2. 以 P 為 baseline：

$$S_{Q/P} = \frac{T_P}{T_Q} = \frac{0.225}{0.192857} \approx 1.16667.$$

時間減少比例是

$$\frac{T_P - T_Q}{T_P} = \frac{0.225 - 0.192857}{0.225} \approx 14.286\%.$$

「Q 快 1.167 倍」與「Q 少用 14.286% 時間」描述同一比較。不能把 1.167 寫成「少用 16.7% 時間」。

3. Q 的 CPI 升到 2.10：

$$T'_Q = \frac{(3.0 \times 10^8)(2.10)}{2.8 \times 10^9} = 0.225\ \text{s}.$$

$T'_Q = T_P$，所以兩者同速。不過這不是 instruction count 與 CPI「互相抵銷」：Q 的 cycle 數由 P 的 5.4×10^8 增為 6.3×10^8，是較高的 2.8 GHz frequency 把較多的 cycles 壓回相同時間。若兩者都用 2.4 GHz，Q 會慢 16.667%。

4. Q 的 frequency 2.4 GHz、CPI 1.40：

$$T''_Q = \frac{(3.0 \times 10^8)(1.40)}{2.4 \times 10^9} = 0.175\ \text{s}, \qquad S = 0.225/0.175 \approx 1.28571.$$

Compiler 可能同時改變 IC 與 instruction mix，進而影響 CPI；microarchitecture 也影響停滯、CPI 與可用頻率。三者必須放入同一時間式比較。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH01-Q03

1. 總 instructions：

$$IC = 2.0 \times 10^9 + 0.5 \times 10^9 + 0.5 \times 10^9 = 3.0 \times 10^9.$$

總 cycles：

$$C = 2.0(1) + 0.5(4) + 0.5(3) = 2.0 + 2.0 + 1.5 = 5.5 \times 10^9.$$

平均 CPI：

$$CPI_{\text{avg}} = 5.5/3.0 \approx 1.83333.$$

At 3 GHz：

$$T = 5.5 \times 10^9/(3.0 \times 10^9) \approx 1.83333 \text{ s}.$$

若 load/store CPI 由 4 降為 3，cycle 數減少 0.5×10^9：

$$C' = 5.0 \times 10^9, \quad T' = 1.66667 \text{ s}, \quad S = 5.5/5.0 = 1.10.$$

2. S1 有 $2 + 1 + 2 = 5$ 條 instructions：

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \qquad CPI_1 = 10/5 = 2.0.$$

S2 有 $4 + 1 + 1 = 6$ 條 instructions：

$$C_2 = 4(1) + 1(2) + 1(3) = 9, \qquad CPI_2 = 9/6 = 1.5.$$

相同 frequency 下 $T_1 > T_2$，S2 speedup 為

$$S_{2/1} = C_1/C_2 = 10/9 \approx 1.111.$$

S2 多 20% instructions，但每條平均較少 cycles；總 cycles 才是共同分母。

3. 只看 instruction count 會選 S1，結論卻是錯的。S1 的 5 條少於 S2 的 6 條，但 S1 要 10 cycles，S2 只要 9 cycles。必須同時看 instruction mix 與 CPI。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH01-Q04

1. 令 baseline $T_0 = 1$：

$$T_1 = (1 - 0.72) + \frac{0.72}{6} = 0.28 + 0.12 = 0.40,$$
$$S = 1/0.40 = 2.5.$$

2. $s \to \infty$ 時：

$$T_\infty = 0.28, \qquad S_\infty = 1/0.28 = 25/7 \approx 3.57143.$$

3. 在有限的 s 下，$\frac{0.72}{s} > 0$，所以 $T_1 > 0.28$、$S < 25/7$。極限是任意有限加速都只能逼近而不可達的上界；若成本也不同，實際設計更不能把極限當 measured result。

4. 目標 20 s：

$$20 = \frac{80}{s} + 20 \quad \Rightarrow \quad \frac{80}{s} = 0.$$

沒有任何 finite s 滿足；$s \to \infty$ 也只會趨近 20 s。目標 50 s 時：

$$50 = \frac{80}{s} + 20 \quad \Rightarrow \quad 30s = 80 \quad \Rightarrow \quad s = 8/3 \approx 2.667.$$

CH01-Q05

1. 無 pipeline 的吞吐量是

$$\frac{1}{0.020\ \text{s}} = 50\ \text{frames/s}.$$

有 pipeline 的穩定完成間隔是 8 ms，吞吐量是

$$\frac{1}{0.008\ \text{s}} = 125\ \text{frames/s}.$$

2. 假設三個 stage 均勻且 frame 在 phase 邊界完成，第一張在 $t = 24$ ms 完成。第二張晚一個完成間隔，在 $t = 32$ ms 完成。

3. 單張 frame 的端到端 latency 從 20 ms 變 24 ms，因此若只在意單張完成時間，改動不利。Pipeline 的吞吐優勢要累積多張 frame 才能實現。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH01-Q06

1. X：

$$GM_X = (1.44 \cdot 1.00 \cdot 0.81)^{1/3} \approx 1.0526.$$

Y：

$$GM_Y = (1.20 \cdot 1.25 \cdot 1.08)^{1/3} \approx 1.1745.$$

2. $GM_Y > GM_X$，Y 的等權幾何平均較高。
3. 任意另一台 reference machine 的程式 i 時間記為 $T^{\star}_{\text{ref},i}$。新 ratio 是

$$r^{\star}_{X,i} = T^{\star}_{\text{ref},i}/T_{X,i}, \qquad r^{\star}_{Y,i} = T^{\star}_{\text{ref},i}/T_{Y,i}.$$

兩者相除：

$$r^{\star}_{X,i}/r^{\star}_{Y,i} = \frac{T_{Y,i}}{T_{X,i}},$$

於是

$$\frac{GM^{\star}_X}{GM^{\star}_Y} = \left(\prod_i \frac{T_{Y,i}}{T_{X,i}}\right)^{1/n} = \frac{GM_X}{GM_Y}.$$

每個 $T^{\star}_{\text{ref},i}$ 都成對抵銷，甚至不需要它們按同一比例改變。若用算術平均，$\sum_i T^{\star}_{\text{ref},i}/T_{X,i}$ 對 $\sum_i T^{\star}_{\text{ref},i}/T_{Y,i}$ 的比值會依新 reference 的各程式時間而改變，不能提供同樣的 invariant。

4. 幾乎全是 R1 時，X 的 1.44 高於 Y 的 1.20，選型應依 R1 或重新加權。等權 geometric mean 把 R2/R3 也納入，只能描述那組 benchmark 的摘要，不能代表所有 workloads。

CH01-Q07

1. 一階模型下 $P \propto V^2 f$。若 voltage 與 frequency 都乘 0.8，

$$P_{\text{new}} = 40(0.8^2)(0.8) = 40(0.512) = 20.48 \text{ W}.$$

2. cycle 數不變，frequency 成為 0.8 倍，所以時間成為 $1/0.8 = 1.25$ 倍：

$$t_{\text{new}} = 500(1.25) = 625 \text{ ms}.$$

3. Energy 是平均 power 乘時間，在一階固定功率近似下，

$$E_{\text{new}} = 20.48 \text{ W} \times 0.625 \text{ s} = 12.8 \text{ J}.$$

原 energy 是

$$E_{\text{old}} = 40 \text{ W} \times 0.5 \text{ s} = 20 \text{ J}.$$

降低比例為

$$\frac{20 - 12.8}{20} = 0.36 = 36\%.$$

CH01-Q08

1. Parser 占 baseline 的 0.15。即使 parser 無限加速，樂觀 latency 上限是

$$S_{\text{latency,max}} = \frac{1}{1 - 0.15} = \frac{1}{0.85} \approx 1.176.$$

若只加速 3 倍，而其他部分時間不變，

$$T_{\text{new}} = 0.85 + \frac{0.15}{3} = 0.90,$$

故 latency speedup 是 $1/0.90 \approx 1.111$。

2. Throughput 由系統中最擁擠的共享資源與服務流程決定。Parser 加速雖縮短部分 CPU 工作，但 memory bandwidth 接近飽和，其他 pipeline stages 無法比例增加完成 requests。CPU 利用率上升可能包含排程中執行的忙等或記憶體停滯，不代表有效 work 增加；阻塞且未排程的等待通常不計入 CPU 利用率。

3. 先量測 memory bandwidth 的使用率與 queueing latency，再量測每個 service stage 的 utilization 與 latency。這兩類證據可區分「共享記憶體飽和」與「其他序列 stage 排隊」。同時確認 requests 是否有相同 arrival rate 與 input distribution。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 1. CSO course slides, 2026, Chapter 1.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study